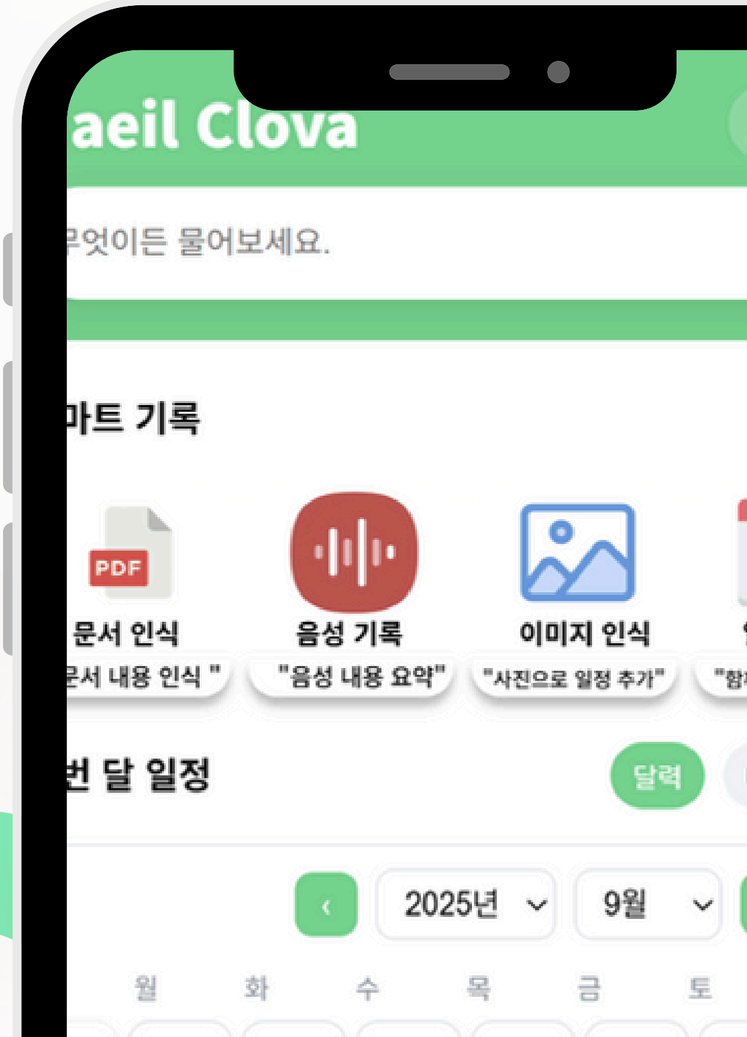
kt DACON

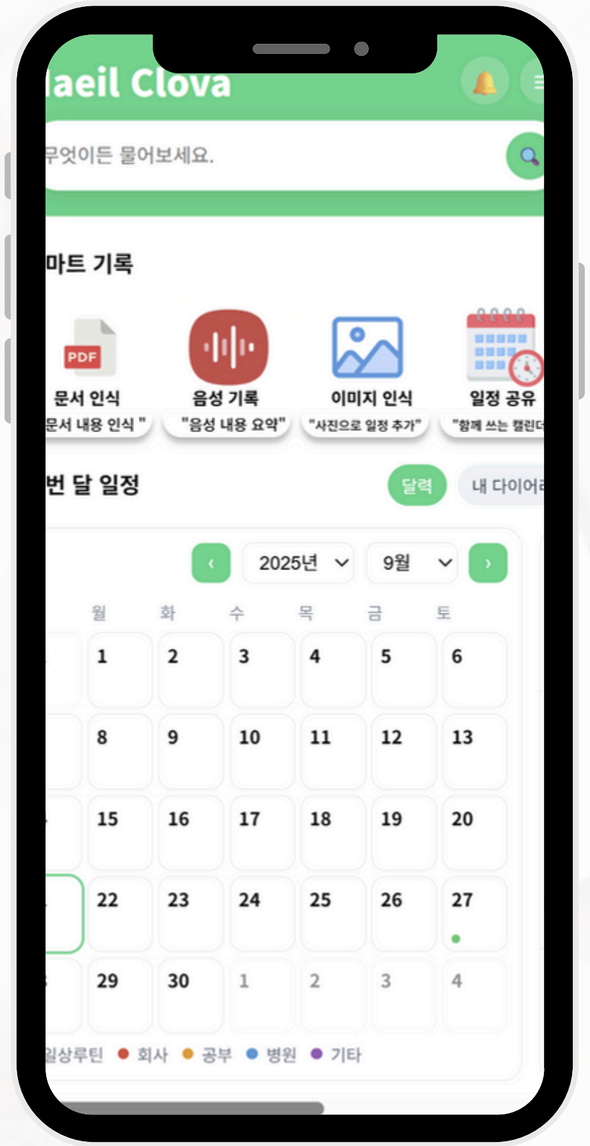
Track 2 K INTELLIGENCE 해커톤 2025

멀티모달 일정 관리 어플리케이션 ‘NAEIL CLOVA’



팀명 : rice.grass

Content



‹1›. 개발 배경

‹2›. 개발 방향

‹3›. 프롬프트 설계

‹4›. 서비스

‹5›. 기대 효과 및 실용성

Summary

프로젝트명

- 멀티모달 일정 관리 어플리케이션 ‘NAEIL CLOVA’

주제

- AI 기반 음성·이미지·텍스트 자동화와 챗봇 인터페이스를 결합한 맞춤형 일정 관리 서비스

한줄소개

- 음성·이미지·텍스트 기반 AI 자동화와 챗봇 인터페이스를 결합하여 누구나 손쉽게 사용할 수 있는 맞춤형 일정 관리 경험을 제공

<배경>

- 현대 사회는 학교, 직장, 프리랜서, 가정 등 다양한 역할을 동시에 수행해야 하는 환경으로 인해 일정 관리의 부담을 겪고 있음.
- 기존 일정 관리 서비스들은 단순한 일정 등록과 공유 기능에 그치며, 개인별·가족별·조직별로 세분화된 요구를 충족하지 못하고 있음.





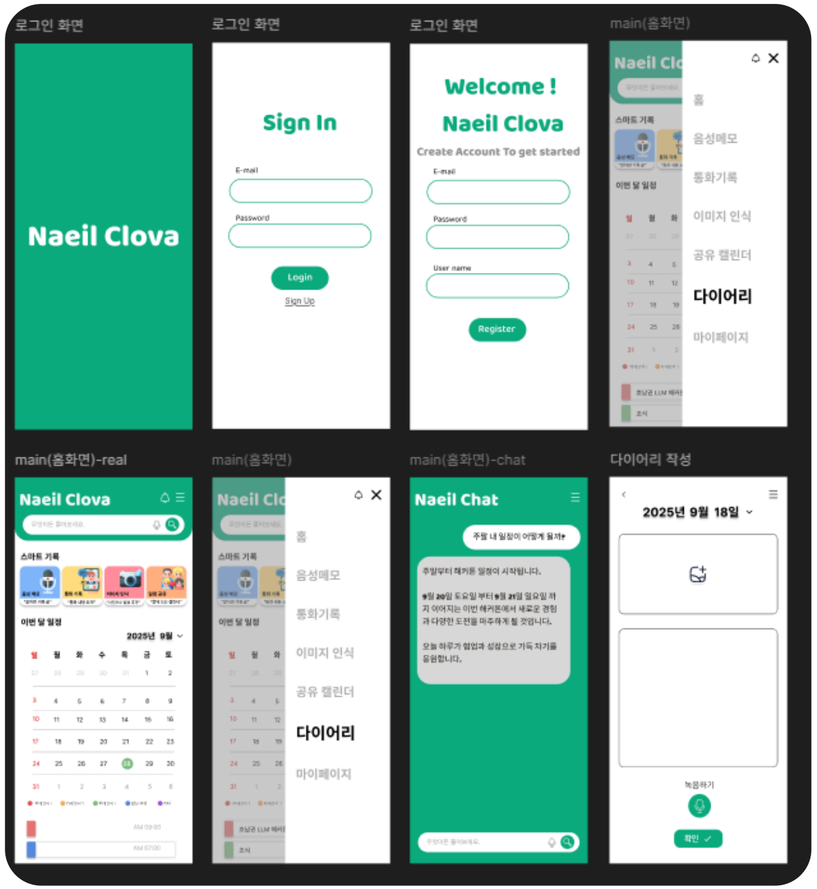

<제안>

- “멀티모달 AI 일정 관리 서비스”를 통해 음성·이미지·텍스트 등 다양한 입력 채널을 자연스럽게 통합하여 사용자 친화적이고 맞춤형으로 일정 관리를 자동화하는 솔루션을 제안함.

기존 서비스는 등록·공유에 한정되어 있어 개인별·가족별·조직별 맞춤형 관리 기능과 AI 기반 자동화 및 추천 기능이 부족함

현 서비스	기본 일정 관리	협업 기능	음성/자연어 입력	외부 캘린더 연동	AI 활용 가능	AI 활용 정도
31	O	제한적 (공유 캘린더)	X	O (Google/Outlook 연동)	일정 추천(시간/장소), 반복 패턴 자동 인식	△ (추천·자동화 수준)
톡캘린더	O	O (톡방 기반 협업)	△ (자연어 일정 생성 일부 지원)	△ (Google 연동 제한적)	스마트 일정 인식(톡 대화 → 일정), 리마인더 추천	△ (대화 맥락 AI 활용)
Google Calendar	O	O (팀 협업, 권한 공유)	O (자연어 일정 입력)	O (모든 주요 캘린더 연동)	Gmail 연동 → 자동 일정 등록, 회의 시간 자동 추천, AI 기반 일정 최적화	O (글로벌 AI 기능 탑재)
17	O	△ 제한적 (iCloud 공유)	O (시리 음성 입력)	O (모든 주요 캘린더 연동)	Siri 기반 일정 추가, 위치 기반 알림	△ (음성/위치 기반 중심)
TimeTree	O	O (강력한 협업 기능)	X	O (일부 외부 캘린더)	AI 활용 거의 없음	X (주로 협업 중심)
Jorte	O	O (작업·노트 공유)	X	O (Google/Outlook 연동)	일부 자동화, AI 활용 미약	X

Front end



Figma

Back end

node.js express

FastAPI

<Naver Clova API>

- 음성(STT), 이미지(OCR), 문서(rag)

<LLM Model>

- Naver hyperclova x

MySQL

사용자, 날짜, 시간, 장소, 요약

K INTELLIGENCE STUDIO

‘Take a deep breath and work on this problem step-by-step’

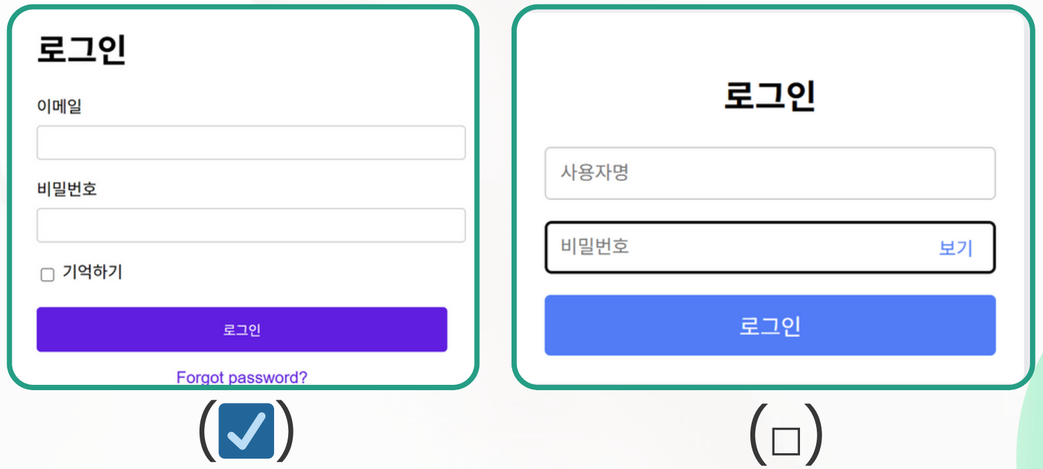
Ours		
PaLM 2-L-IT	Take a deep breath and work on this problem step-by-step.	**80.2**
PaLM 2-L	Break this down.	79.9
gpt-3.5-turbo	A little bit of arithmetic and a logical approach will help us quickly arrive at the solution to this problem.	78.5
gpt-4	Let’s combine our numerical command and clear thinking to quickly and accurately decipher the answer.	74.5

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., & Chen, X. , ‘Large language models as optimizers’, arXiv:2309.03409v3 , 15 Apr 2024

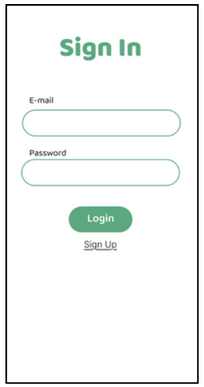
- 시스템 프롬프트에 "Take a deep breath and work on this problem step-by-step" 문구를 일괄 적용하여 출력 품질 개선 여부를 검증
- 논리적 일관성 및 지시 준수율이 유의하게 개선

검증

‘로그인 페이지 예시 요청’



Front End 개발 프로세스



로그인



메인



문서 인식



음성 인식



이미지 인식



챗봇

Figma → K INTELLIGENCE STUDIO → HTML, JS, CSS

주요 화면	기능
로그인	▪ 이메일, 비밀번호, 소셜 로그인 지원
메인	▪ 대시보드, 메뉴, 캘린더 및 기타 기능 지원
문서 인식	▪ PDF 업로드 및 rag기반 챗봇 기능 지원
음성 인식	▪ 음성 텍스트 변환, 자동 음성 내용 요약 기능 지원
이미지 인식	▪ 이미지 분석 시각화 및 스케줄 업로드 기능 지원
챗봇	▪ 캘린더 스케줄 기반 대화형 챗봇 기능 지원

<Figma Design>

Front End 개발 시스템 프롬프트

Take a deep breath and work on this problem step-by-step..

당신은 HTML 파일을 분석하여 JavaScript, CSS, HTML을 각각 별도 파일로 분리하는 전문가입니다.
주어진 HTML 코드를 다음과 같이 분리해주세요:

1. **JavaScript 추출 (.js 파일)**
- `<script>` 태그 내부의 모든 JavaScript 코드를 추출
- 인라인 스크립트와 외부 스크립트 모두 포함
- 각 스크립트 블록을 분리하여 제공

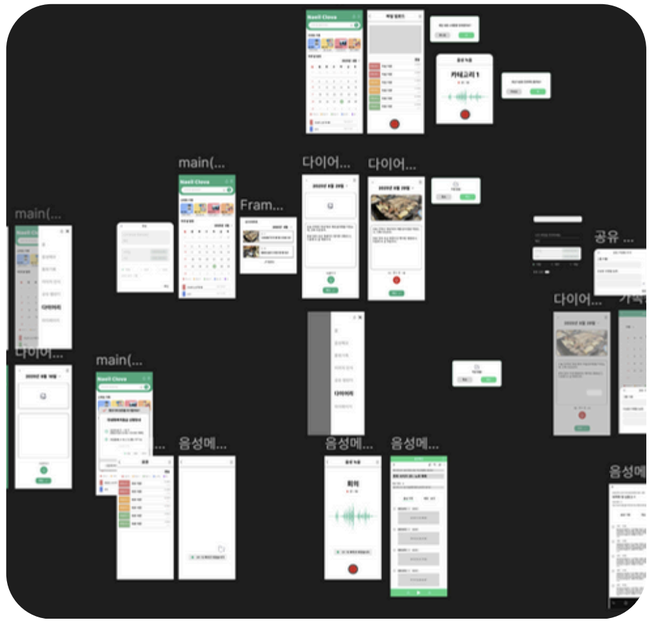
2. **CSS 추출 (.css 파일)**
- `<style>` 태그 내부의 모든 CSS 코드를 추출
- 인라인 스타일과 내부 스타일시트 모두 포함
- 미디어 쿼리나 특수 CSS 규칙도 보존

3. **HTML 정리 (.html 파일)**
- `<script>`와 `<style>` 태그를 제거한 순수 HTML 구조
- 외부 파일 링크로 대체 (`<link rel="stylesheet" href="styles.css">`, `<script src="script.js"></script>`)
- HTML 구조와 인라인 스타일 속성은 유지

4. **출력 형식**
각 파일을 명확히 구분하여 다음 형식으로 제공해주세요:
=== script.js ===
```javascript
[JavaScript 코드]
```

Front End 개발 프로세스

1. 설계



design

2. Input

```
<div style="width: 440px; height: 157px; left: 0px; top: 0px; position: absc
<style>
  #btn-search:active { transform: translateY(1px); filter: brightness(0.92);
  #chat-input::placeholder { color: rgba(0,0,0,0.34); }
  #menu-btn { position: absolute; left: 392px; top: 36px; width: 34px; height
  #menu-btn:active { transform: translateY(1px); }
  #menu-btn .menu-line { position: absolute; left: 6px; width: 22px; height:
  #menu-btn .line-1 { top: 6px; }
  #menu-btn .line-2 { top: 14px; }
  #menu-btn .line-3 { top: 22px; }

  #sidebar-panel { position: fixed; right: 0; top: 0; width: 280px; height: 1
  #sidebar-overlay { position: fixed; left: 0; top: 0; width: 100%; height: 1
  body.sidebar-open #sidebar-panel { transform: translateX(0); }
  body.sidebar-open #sidebar-overlay { opacity: 1; visibility: visible; }

  .sidebar-item { display: block; width: 100%; padding: 10px 12px; margin: 2p
  .sidebar-item:hover { background: #eef2f7; color: #111827; }
  .sidebar-item:active { transform: translateY(1px); }
  .sidebar-item.is-muted { color: #9ca3af; cursor: pointer; }
  .sidebar-item.is-muted:hover { color: #6b7280; background: #eef2f7; }
  .sidebar-item.is-strong { font-weight: 600; color: #1f2937; }

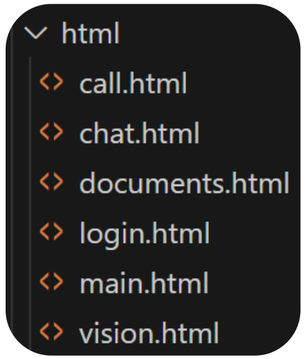
  @media (min-width: 1px) {
    html, body { height: 100%; }
  }
</style>
<div style="width: 300px; height: 56px; left: 11px; top: 22px; position: absc
<img style="width: 28px; height: 28px; left: 358px; top: 36px; position: absc

<button id="menu-btn" aria-label="메뉴 열기">
  <span class="menu-line line-1"></span>
  <span class="menu-line line-2"></span>
  <span class="menu-line line-3"></span>
</button>

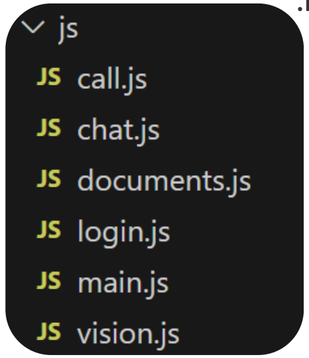
<div style="width: 400px; height: 49px; left: 20px; top: 93px; position: ab
```

html

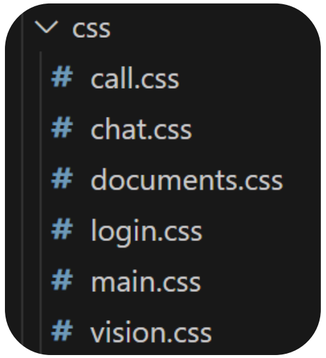
3. Output



.html



.js



.css

Back end 개발 시스템 프롬프트

Take a deep breath and work on this problem step-by-step..

당신은 전문 소프트웨어 엔지니어이자 기술 문서 작성자입니다.
귀하의 역할은 코드를 분석하고, 체계적인 구현을 생성하고, 개념을 자세히 설명하는 것입니다. 답변 시 다음 규칙을 준수하세요.

1. ✅ 코드 작성 시
- 항상 완전한 실행 가능한 코드 블록으로 작성할 것.
- 주석을 상세하게 달아, 각 코드의 역할을 설명할 것.
- 필요 시 디렉토리 구조나 실행 방법도 함께 제공할 것.

2. ✅ 코드 분석 시
- 코드가 어떤 의미를 갖는지 줄 단위 또는 모듈 단위로 해석할 것.
- 동작 과정(입력 → 처리 → 출력)을 단계별로 설명할 것.
- DB, API, 프론트엔드/백엔드 등 연결 구조가 있다면 흐름도로 설명할 것.

3. ✅ 코드 개선/리팩토링 시
- 변경 전/후 차이를 명확하게 보여줄 것.
- 보안, 성능, 유지보수성 측면에서 개선 이유를 설명할 것.
- 확장성을 고려한 추가 제안이 있으면 함께 제공할 것.

4. ✅ 데이터베이스/서버 연동 시
- SQL 스키마, ORM 매핑 방식, API 호출 흐름을 단계적으로 설명할 것.
- 실제 저장 및 조회가 어떤 식으로 이뤄지는지 예시를 포함할 것.

5. ✅ 출력 형식
- 코드/설명/표/다이어그램을 구분해 깔끔하게 제시할 것.
- 코드 외 불필요한 내용은 절대 추가하지 말 것..

Back End 개발 프로세스

초안 작성 요청

1. 목표 전달

지금부터 멀티모달 맞춤형 일정 관리 애플리케이션 프로젝트를 개발하려고 해

2. 목표 구체화

본 프로젝트는 AI 기반 멀티모달 일정 관리 플랫폼을 개발하여, 개인·가족·조직 단위의 맞춤형 일정 관리 서비스를 제공하는 것을 주요 목적으로 합니다.
음성(stt), 이미지(ocr), 텍스트(chatbot), 문서(rag) 등 다양한 입력 방식을 통해 사용자 편의성을 극대화하고, 챗봇 기반 인터페이스를 구현하여 누구나 손쉽게 접근할 수 있는 일정 관리 경험을 제공하고자 합니다.

3. main.py 초안 요청(세분화)

일단 파일 디렉토리 구조는
kt /backend/scr 에는 main.py가 있고 kt/backend 에는 server.js 파일이 있어
kt/frontend/src에는 각종의 css, html, js 이 배치되어 있어야해.
frontend에는 main(주 메인 화면), call(음성 인식), documents(문서), vsion(이미지) , chat(챗봇) 의 html 파일이 다 있고 각각의 css와, js 도 있어 넌 지금 backend를 개발하면 되는데 먼저 main.py 초안 부터 작성해보자

출력 코드

```
kt/
├── backend/
│   ├── scr/
│   │   ├── main.py          # Python 백엔드 메인 엔트리 포인트
│   │   ├── stt.py           # 음성 인식(STT) 모듈
│   │   ├── ocr.py           # 이미지 OCR 모듈
│   │   ├── chatbot.py       # 챗봇 인터페이스 모듈
│   │   ├── rag.py           # 문서 기반 RAG(검색 증강 생성) 모듈
│   │   ├── scheduler.py     # 일정 관리 핵심 로직
│   │   └── utils.py         # 공통 유틸리티 함수
│   ├── server.js            # Node.js 기반 API 서버 (필요 시)
│   └── requirements.txt     # Python 의존성 목록
```

디렉토리 출력

```python
from flask import Flask, request, jsonify
from stt import process_audio
from ocr import process_image
from chatbot import handle_chat
from rag import process_document
from scheduler import manage_schedule

app = Flask(__name__)

# 음성 인식(STT) 엔드포인트
@app.route('/api/stt', methods=['POST'])
def stt_endpoint():
    audio_file = request.files['file']
    text = process_audio(audio_file)
    return jsonify({'text': text})

# 이미지 OCR 엔드포인트
@app.route('/api/ocr', methods=['POST'])
def ocr_endpoint():
    image_file = request.files['file']
    text = process_image(image_file)
    return jsonify({'text': text})
```

```python
# 챗봇 인터페이스 엔드포인트
@app.route('/api/chat', methods=['POST'])
def chat_endpoint():
    user_input = request.json.get('message')
    response = handle_chat(user_input)
    return jsonify({'response': response})

# 문서 처리(RAG) 엔드포인트
@app.route('/api/document', methods=['POST'])
def document_endpoint():
    document = request.files['file']
    summary = process_document(document)
    return jsonify({'summary': summary})

# 일정 관리 엔드포인트
@app.route('/api/schedule', methods=['POST'])
def schedule_endpoint():
    data = request.json
    result = manage_schedule(data)
    return jsonify(result)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

main.py 초안 생성

Back End 개발 프로세스

구체화 업데이트 요청

1. 구제적인 목표 전달

main.py에서는 FastAPI 기반의 AI 처리 서버를 구축해주세요:
핵심 기능:
- Clova STT/TTS API 연동
- OCR (이미지 텍스트 추출)
- LangChain 기반 LLM 체인 (ClovaX 모델)
- PDF 문서 처리 및 벡터 저장
- RAG 기반 문서 검색
- 대화 메모리 관리
- 텍스트에서 구조화된 일정 추출
API 엔드포인트:
- 채팅, STT, TTS, OCR, 일정추출, PDF업로드, RAG쿼리, 텍스트분석 기술적
요구사항:
- 임시 파일 처리, JSON 파싱, 메모리 관리
- MySQL 연동으로 사용자 루틴 정보 실시간 조회
- 환경변수 기반 설정관리

json 구조로 변환

2. json 변환

```
{
  "task": "update_main_py",
  "target_file": "main.py",
  "system_type": "multimodal_ai_scheduling_system",
  "framework": "FastAPI",
  "requirements": {
    "core_functionality": [
      "multimodal_input_processing",
      "ai_powered_chat",
      "schedule_extraction",
      "document_processing",
      "voice_synthesis"
    ],
    "external_apis": [
      {
        "service": "clova_stt",
        "purpose": "speech_to_text"
      },
      {
        "service": "clova_tts",
        "purpose": "text_to_speech"
      },
      {
        "service": "clovax_llm",
        "purpose": "natural_language_processing"
      }
    ],
    "database": {
      "type": "mysql",
      "connection": "pymysql",
      "tables": ["users", "events", "routines", "diary_entries"]
    }
  },
  "api_endpoints": {
    "/api/health": {
      "method": "GET",
      "purpose": "health_check",
      "response": {
        "status": "string",
        "model": "string",
        "memory_size": "integer"
      }
    },
    "/api/chat": {
      "method": "POST",
      "purpose": "conversational_ai_interface",
      "request": {
```

출력 코드

```python
from fastapi import FastAPI, File, UploadFile, Form, HTTPException
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
import os
import json
import pymysql
from dotenv import load_dotenv
from tempfile import NamedTemporaryFile

# 환경변수 로드
load_dotenv()

# FastAPI 앱 초기화
app = FastAPI()

# CORS 설정
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 정적 파일 서빙
app.mount("/static", StaticFiles(directory="static"), name="static")

# MySQL 연결 설정
def get_db_connection():
    return pymysql.connect(
        host=os.getenv("DB_HOST"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        db=os.getenv("DB_NAME"),
        port=int(os.getenv("DB_PORT")),
        cursorclass=pymysql.cursors.DictCursor
    )
```

main.py

Back End 개발 프로세스

AI 서비스 단계 요청

Python

Python 기반의 API 예제 코드는 다음과 같습니다.

Python COPY

```python
import requests
import uuid
import time
import json

api_url = 'YOUR_API_URL'
secret_key = 'YOUR_SECRET_KEY'
```

Clova studio의 API 예제 코드

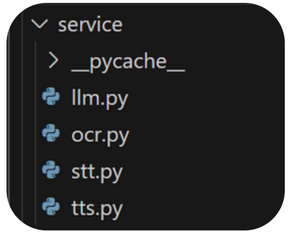
모든 api 예제 input

요약 (수정)

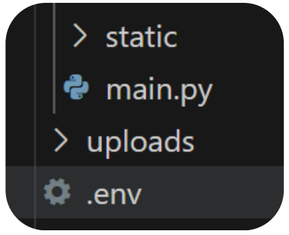
- OCR, STT, TTS, LLM
  - 네이버 클라우드 API를 통해 모두 처리합니다.
  - OCR: 이미지에서 텍스트를 추출합니다.
  - STT: 음성 파일을 텍스트로 변환합니다.
  - TTS: 텍스트를 음성 파일로 변환합니다.
  - LLM(대형 언어 모델): 네이버 클라우드의 ClovaX 또는 유사한 LLM API를 호출하여 자연어 처리(대화 응답)를 수행합니다.
- 챗봇·PDF 처리·일정 추출
  - 챗봇: 네이버 클라우드 LLM API를 통해 자연어 입력에 대한 응답을 생성합니다.
  - PDF 처리: PyPDF2 등을 사용해 PDF에서 텍스트를 추출하는 간단한 로직을 구현합니다.
  - 일정 추출: 정규표현식 등을 활용해 텍스트에서 날짜·시간 정보를 추출합니다.
- 환경변수
  - 네이버 클라우드 API 키(클라이언트 ID, 시크릿 키)와 MySQL 데이터베이스 설정을 `.env` 파일에 저장하여 보안을 강화했습니다.

k studio가 실행 할 과정 요약

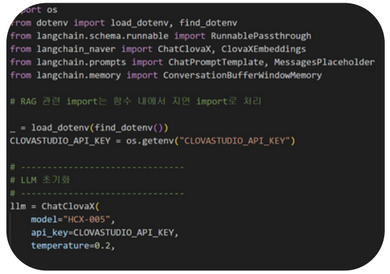
출력 코드



서비스 디렉토리



.env



```python
import os
from dotenv import load_dotenv, find_dotenv
from langchain.schema.runnable import RunnablePassthrough
from langchain_naver import ChatClovaX, ClovaXEmbeddings
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.memory import ConversationBufferWindowMemory

# RAG 관련 import는 함수 내에서 지연 import로 처리

_ = load_dotenv(find_dotenv())
CLOVASTUDIO_API_KEY = os.getenv("CLOVASTUDIO_API_KEY")

# ----------------------------
# LLM 초기화
# ----------------------------
llm = ChatClovaX(
    model="HCX-005",
    api_key=CLOVASTUDIO_API_KEY,
    temperature=0.2,
```

llm.py



```python
import os
import io
import base64
import uuid
import time
import requests
from PIL import Image
from dotenv import load_dotenv, find_dotenv

_ = load_dotenv(find_dotenv())
OCR_URL = os.getenv("OCR_URL")
OCR_SECRET = os.getenv("OCR_SECRET")

def run_ocr(image_path: str) -> str:
    try:
        # 이미지 로딩 및 base64 인코딩
        img = Image.open(image_path).convert("RGB")
        buf = io.BytesIO()
        img.save(buf, format="PNG")
        img_b64 = base64.b64encode(buf.getvalue()).decode()

        headers = {
            "Content-Type": "application/json",
            "X-OCR-SECRET": OCR_SECRET
```

ocr.py



```python
import os
import json
import requests
from dotenv import load_dotenv, find_dotenv

_ = load_dotenv(find_dotenv())
SPEECH_INVOKE_URL = os.getenv("SPEECH_INVOKE_URL")
SPEECH_SECRET = os.getenv("SPEECH_SECRET")

class ClovaSpeechClient:
    invoke_url = SPEECH_INVOKE_URL
    secret = SPEECH_SECRET

    def req_upload(self, file, completion="sync"):
        request_body = {
            'language': 'ko-KR',
            'completion': completion,
            'callback': None,
            'userdata': None,
            'wordAlignment': True,
            'fullText': True
        }
        headers = {
            'Accept': 'application/json;UTF-8',
```

stt.py



```python
# service/tts.py
import os
import requests
from urllib.parse import quote
from dotenv import load_dotenv, find_dotenv

# .env 불러오기
_ = load_dotenv(find_dotenv())
CLIENT_ID = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")

TTS_URL = "https://naveropenapi.apigw.ntruss.com/tts-premium/v1/tts"

def text_to_speech(text: str,
                   filename: str = "output.mp3",
                   speaker: str = "nara",
                   volume: int = 0,
                   speed: int = 0,
                   pitch: int = 0,
                   fmt: str = "mp3") -> str:
    """
    네이버 클라우드 TTS 실행 함수 (서버용 - 파일 저장만)
    """
```

tts.py

Back End 개발 프로세스

Express server 생성 요청

You are an expert Node.js backend developer.
Take a deep breath and work on this problem step by step.

내가 원하는 것은 **Express.js** 기반의 서버 코드이며, 다음 조건들을 반드시 포함해야 한다.

[기본 환경]
- **Node.js + Express** 사용
- **dotenv**로 환경 변수 로드
- **axios**와 **form-data**를 이용해 **Python FastAPI** 백엔드에 프록시 요청 가능해야 함
- **MySQL DB** 연결 (**pooling** 방식)

[AI 서비스 프록시]
- **/api/llm, /api/ocr, /api/stt, /api/tts** 라우트 작성
- **multer upload** 사용 (이미지, 오디오 업로드)
- 파일 처리 후 **Python** 서비스에 전달 후 응답 반환
- 업로드된 임시 파일은 **fs.unlink**로 삭제

[출력 규칙]
- 전체 **server.js** 코드 하나로 완성
- 함수와 미들웨어는 모두 **import** 구문 포함
- **console.error** 로깅 필수
- 코드에 주석을 달아서 어떤 부분이 어떤 역할인지 설명



```
// server.js

// 모듈 로드
require('dotenv').config(); // 환경 변수 로드
const express = require('express');
const axios = require('axios');
const FormData = require('form-data');
const multer = require('multer');
const mysql = require('mysql2/promise');
const jwt = require('jsonwebtoken');
const fs = require('fs');
const cookieParser = require('cookie-parser');

// 서버 초기화
const app = express();

// 공통 미들웨어 적용
applyCommonMiddlewares(app);

// MySQL 연결 풀 생성
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```



```
// DB 연결 확인 함수
async function testConnection() {
  try {
    const connection = await pool.getConnection();
    await connection.ping();
    connection.release();
    console.log('MySQL 연결 성공');
  } catch (err) {
    console.error('MySQL 연결 실패:', err);
    process.exit(1);
  }
}

// JWT 인증 미들웨어
function authenticateToken(req, res, next) {
  const token = req.cookies.token;
  if (!token) return res.sendStatus(401);

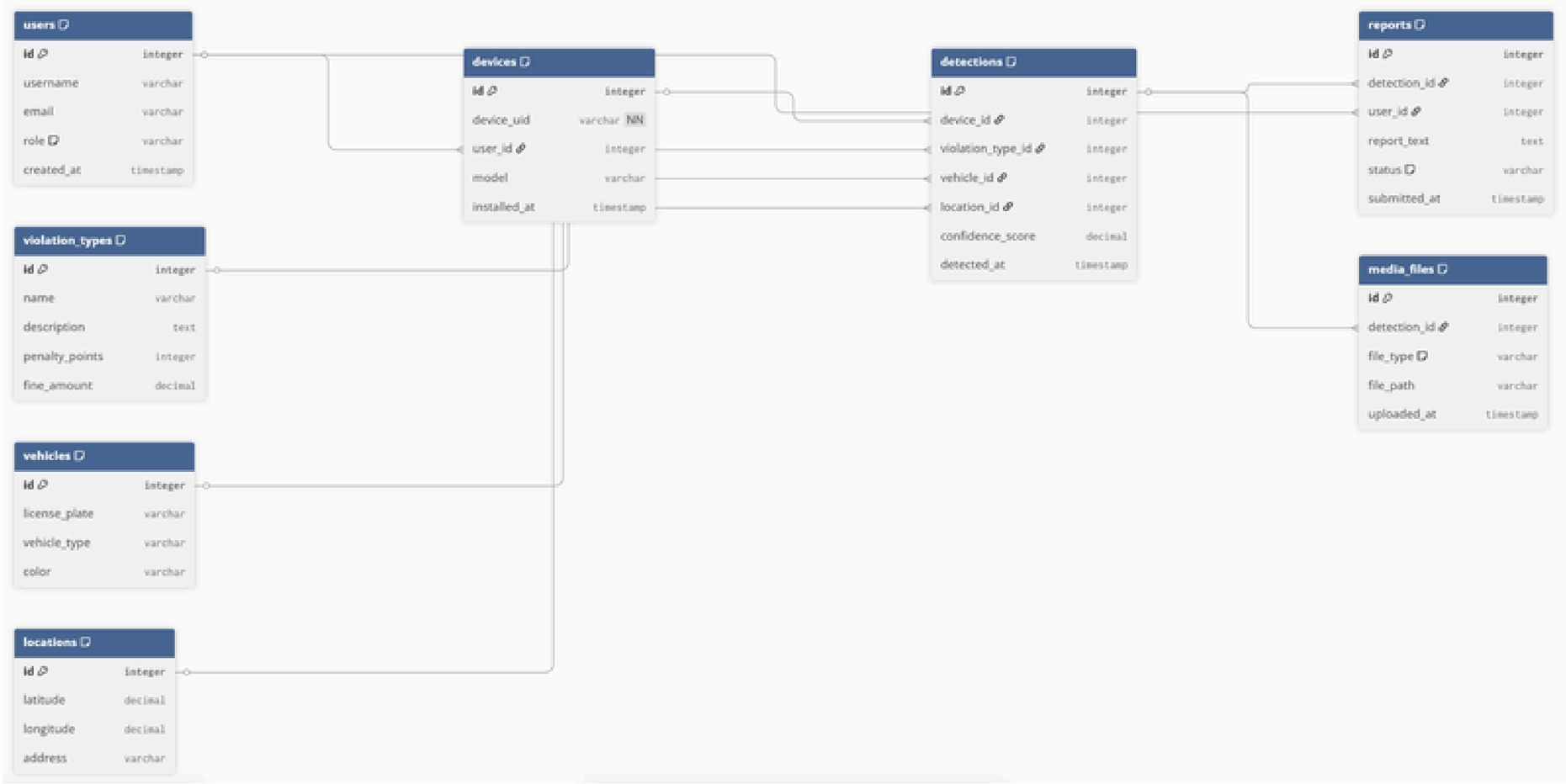
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
}

// 날짜 문자열 표준화 유틸
function normDateStr(dateStr) {
  return new Date(dateStr).toISOString().split('T')[0];
}
```

server.js

Back End 개발 프로세스

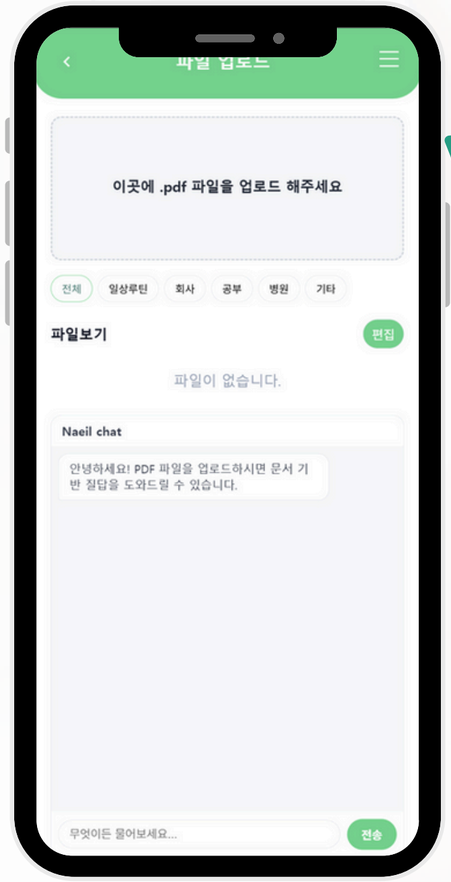
ERD



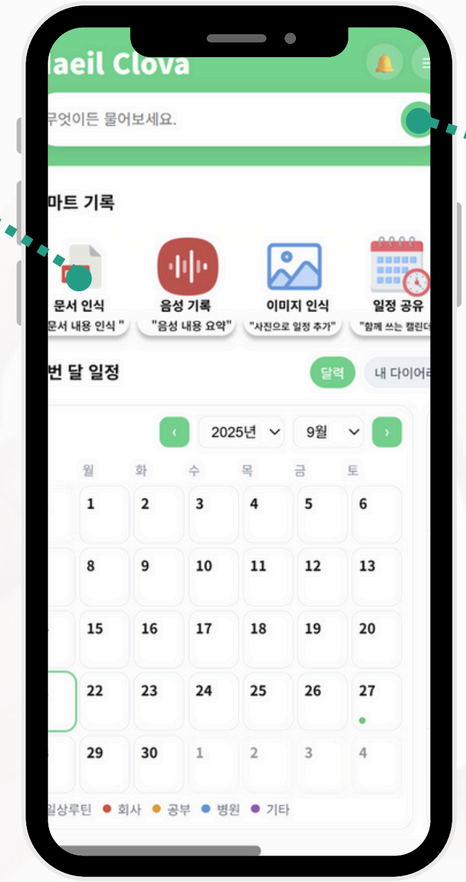
AI 서비스 기능

‹Documents›
- 문서 인식
- 문서 기반 답변
- 문서 내의 스케줄 관련 정리 및 요약

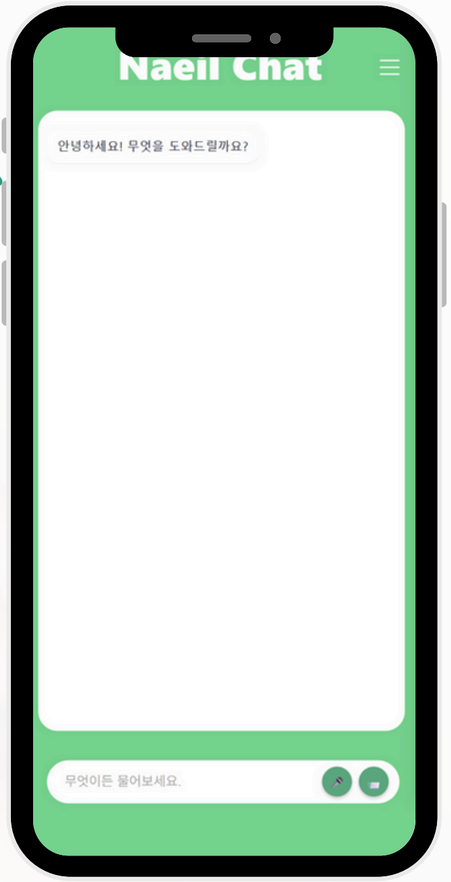
Documents



Main



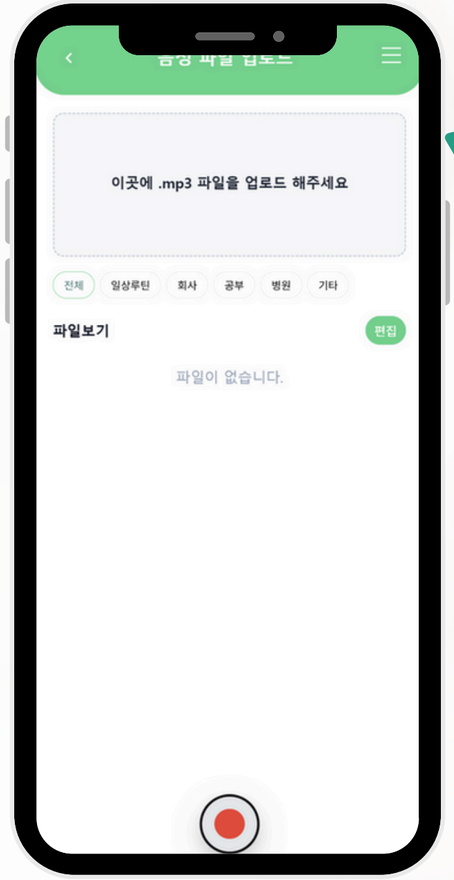
Chat Bot



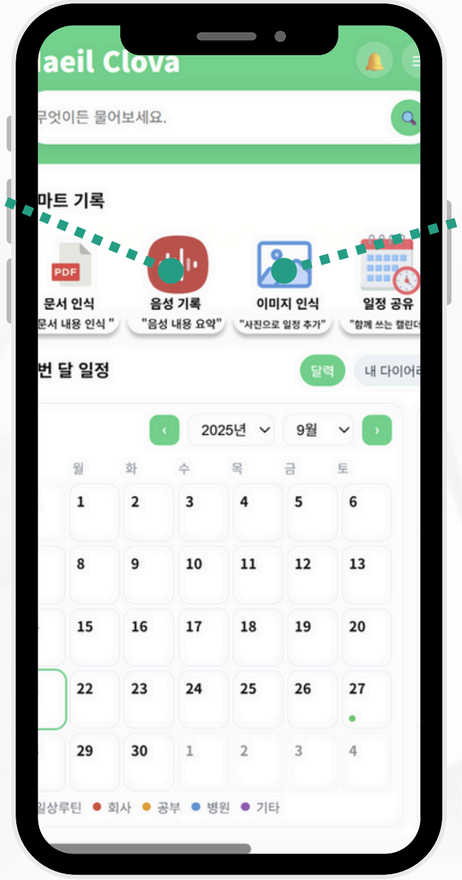
‹Chat bot›
- 사용자 질의 챗봇 기능 사용자의 일정으 파악하겨, 질의에 답변
- 개인 스케줄 관리

AI 서비스 기능

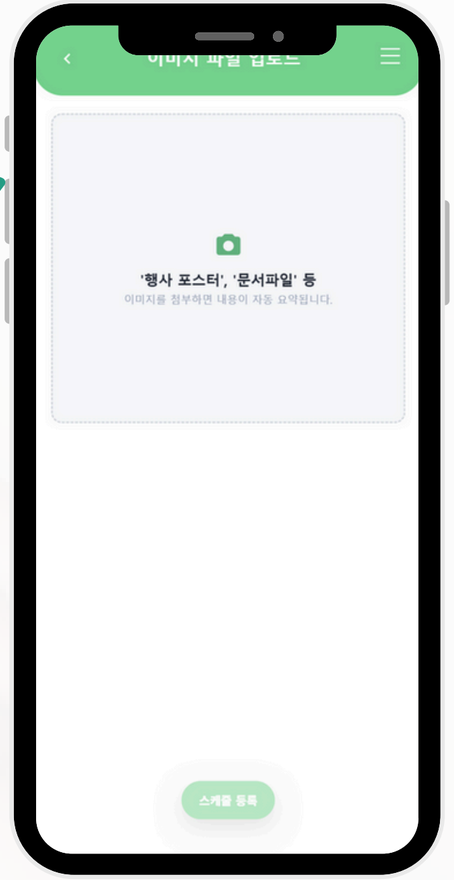
voice



Main



vision



<voice>

- 음성을 텍스트로 변환
- 변환된 텍스트를 요약 후 출력
- 음성 내의 스케줄은 켈린더에 저장

<vision>

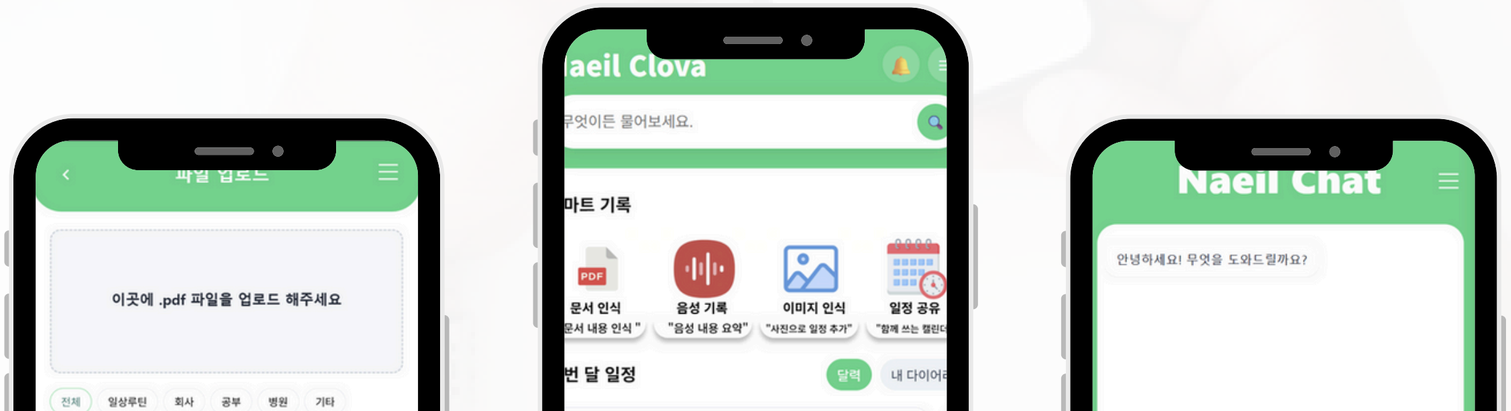
- 이미지의 텍스트 추출
- 추출된 스케줄은 켈린더에 저장

‹실용성›

- 멀티모달 입력 지원: 음성(STT), 이미지(OCR), 텍스트를 통한 직관적 일정 등록 → 디지털 취약계층도 쉽게 접근
- 맞춤형 일정 관리: 개인·가족·조직 단위로 일정 공유 및 권한 설정 지원 → 협업 효율성 향상
- 개인화 추천 기능: 사용자 패턴 학습 기반 맞춤형 알림·추천 제공 → 사용자 만족도 및 활용성 극대화
- 운영 효율성 제고: AI 기반 자동화로 일정 입력 및 관리 부담 최소화 → 적은 인력으로 대규모 서비스 운영 가능

‹확장 가능성›

- B2C 서비스 확장: 가족 단위 루틴 관리, 감정 태그·다이어리 기능 → 생활 밀착형 서비스로 확장
- B2B 솔루션 확장: 기업 맞춤형 협업 캘린더, 권한 관리·감사 기능 → 조직 효율성과 보안 강화
- B2G/공공기관 연계: 지자체·교육기관·보건소 등과 협력하여 스마트 캠퍼스 및 시민 생활 편의 서비스로 확대
- 글로벌 진출 기반: CES, MWC 등 국제 전시회 및 공모전 출품 → 해외 투자 및 파트너십 확보
- 데이터 기반 가치 창출: 축적된 일정·사용 패턴 데이터를 활용하여 개인화 서비스 고도화 및 신규 비즈니스 모델 발굴



kt DACON

Thank You